

INDUSTRIAL TRAINING REPORT

ON

MERN STACK DEVELOPMENT

Training at AccioJob Pvt. Ltd.

Project: AuditPilot – An AI Website Audit Agent

Submitted in partial fulfilment of the academic requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

Computer Science and Engineering (Data Science)

Submitted To:

Dr. Medhavi Malik

Assistant Professor, CSE

Submitted By:

Nikshey Yadav

231302221

B.Tech CSE (DS)

2023–2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING AND TECHNOLOGY

SGT UNIVERSITY, GURUGRAM

November, 2026

CERTIFICATE BY COMPANY

This is to certify that the training report entitled “MERN Stack Development Training at AccioJob Pvt. Ltd.” has been prepared by Nikshey Yadav, a student of B.Tech in Computer Science and Engineering (Data Science), SGT University. The report presents the learning activities, development practices and the project work (AuditPilot, an AI website audit agent) done during the training.

The report is submitted for academic evaluation. The description of training activities is based on the learner’s programme context and should be verified against official training records before final institutional submission.

Training Coordinator: Mr. Subhojit

Name of Student – Nikshey Yadav

Roll. No: - 231302221

Place: - Gurugram, Haryana

Date: 28-10-2026

This is to certify that the above statement made by the candidate is correct to the best of my knowledge: -

Verified by: -

Dr. Medhavi Malik

Assistant Professor

CSE, SOET

SGTU, GURUGRAM

CANDIDATE'S DECLARATION

I, Nikshey Yadav, declare that this training report titled "MERN Stack Development Training at AccioJob Pvt. Ltd." has been prepared for academic purposes and reflects my own understanding of the training topics and of the project work described in it. The report is written in my own words. The books, websites and documents I used for learning are listed in Appendix C.

I also state clearly that I built the AuditPilot project with the help of an AI coding assistant (Claude Code). I took the design decisions, tested the work and can explain every part of it. Express.js, MongoDB and JWT were not part of my classroom training; I learned them while building this project. Details such as mentor names, attendance and marks must be confirmed from official training records.

(Student Signature with Date)

Nikshey Yadav

231302221

(Mentor Signature with Date)

Mr. Jatin

Faculty Mentor, CSE

Signature of the Head of Department

(With Stamp)

HOD CSE

ACKNOWLEDGEMENT

I would like to thank AccioJob Pvt. Ltd. for giving me a clear and well-planned learning path in web development and AI. The training helped me build a base in HTML, CSS, JavaScript and React, and gave me my first lessons in AI, AI agents, Node.js and MCP.

I am thankful to my trainers and mentors for their explanations, examples and feedback. They taught me to break a big problem into small tasks, write clean code and test my work before calling it finished.

I also thank SGT University, my faculty members and my family for their guidance and support. Finally, I am grateful for the free documentation, open-source tools and AI assistance that helped me learn and build my project.

ABSTRACT

This report describes my industrial training at AccioJob Pvt. Ltd. (8 August to 18 March) and the project I built during it. In the training I studied HTML, CSS, JavaScript, React, AI, AI agents, a little Node.js and the basics of MCP. Express.js, MongoDB and JWT were not taught in the training, so I learned them on my own while building the project, with the help of an AI coding assistant.

The project is AuditPilot, an AI website audit agent. A signed-in user enters a website address. Deterministic tools check the site for SEO, accessibility, security and performance problems and produce evidence and scores. A local open-source language model (qwen2.5:3b, run with Ollama) then explains the results, gives a fix plan and answers questions, calling the same tools when it needs more facts. The main idea is: tools compute the facts, the language model only explains them. The app also supports history, sign-up and sign-in, scheduled audits, JavaScript rendering and whole-site audits with a polite crawler.

The report has six chapters. Chapter 1 gives the introduction. Chapter 2 describes the technical topics learnt in the training. Chapter 3 explains how I worked, the tools I used and how I used an AI assistant. Chapter 4 presents the AuditPilot project in detail. Chapter 5 gives the real results, tests, challenges and limits. Chapter 6 gives the conclusion and future scope. All numbers in this report were measured from the running application and its 104 automated tests.

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update and Delete
CSP	Content Security Policy
CSS	Cascading Style Sheets
DNS	Domain Name System
HSTS	HTTP Strict Transport Security
HTML	HyperText Markup Language
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCP	Model Context Protocol
MERN	MongoDB, Express.js, React.js and Node.js
npm	Node Package Manager
OWASP	Open Worldwide Application Security Project
SEO	Search Engine Optimisation
SPA	Single-Page Application
SSE	Server-Sent Events
SSRF	Server-Side Request Forgery
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator
WCAG	Web Content Accessibility Guidelines

Table of Contents

CERTIFICATE BY COMPANY 2

CANDIDATE'S DECLARATION 3

ACKNOWLEDGEMENT 4

ABSTRACT 5

LIST OF ABBREVIATIONS 6

CHAPTER 1: INTRODUCTION 10

1.1 Background of Industrial Training 10

1.2 Overview of MERN Stack Development 10

1.3 Need for Practical Training 11

1.4 Objectives of the Training 11

1.5 Scope of the Report 11

1.6 Learning Outcomes Expected 12

CHAPTER 2: SOFTWARE TRAINING WORK UNDERTAKEN 13

2.1 Training Context and Role 13

2.2 Learning Responsibilities 13

2.3 HTML Learning and Practice 13

2.4 CSS Learning and Practice 13

2.5 JavaScript Learning and Practice 14

2.6 React.js Learning and Practice 14

2.7 AI and AI Agents 14

2.8 Node.js and MCP Basics 14

2.9 Work Discipline and Problem Solving 15

CHAPTER 3: INDUSTRIAL TRAINING WORK UNDERTAKEN 16

3.1 Training Methodology 16

3.2 Daily Learning Workflow 16

3.3 Task Planning and Execution 16
3.4 Debugging and Testing Practices 17
3.5 Communication, Feedback and Documentation 17
3.6 Tools and Development Environment 17
3.7 Professional Skills Developed 18
CHAPTER 4: PROJECT WORK – AUDITPILOT: AI WEBSITE AUDIT AGENT 19
4.1 Project Introduction 19
4.2 Problem Statement 19
4.3 Objectives and Requirements 19
4.4 System Design and Workflow 20
4.5 Audit Engine and Scoring 21
4.6 Security: SSRF Guard and Access Control 21
4.7 AI Agent, Tools and MCP 22
4.8 Database Design 23
4.9 Frontend Implementation 24
4.10 Extra Features 27
4.11 Whole-Site Audit (Crawl) 27
4.12 Testing and Validation 29
4.13 Limitations and Project Learning 29
CHAPTER 5: RESULTS AND DISCUSSIONS 30
5.1 Training Outcomes 30
5.2 Project Results 30
5.3 Technical Observations 31
5.4 Challenges and Solutions 32
5.5 Skills Gained and Self-Assessment 32
5.6 Limitations and Improvement Opportunities 32

CHAPTER 6: CONCLUSION AND FUTURE SCOPE 34
6.1 Conclusion 34
6.2 Future Scope 34
APPENDIX A: PROJECT FEATURE CHECKLIST 35
APPENDIX B: TEST CASES 36
Audit the bad demo page 36
Audit the fixed demo page 36
Compare two audits 36
Block a private address 36
Wrong password 36
Ownership 36
AI model off 36
Chat tool call 37
History search 37
Whole-site audit 37
Sitemap pages 37
APPENDIX C: TECHNOLOGIES AND REFERENCES 38

CHAPTER 1: INTRODUCTION

1.1 Background of Industrial Training

Industrial training connects what we learn in class with real software work. In college we study programming, data structures and computer systems. In training we use these ideas to build, test and improve real software. Training also teaches discipline: finishing work on time, explaining progress and taking feedback in a good way.

In web development there is a big gap between knowing a language and building a working application. A student may know loops and functions but still find it hard to design a page, handle user input, manage data and fix bugs. A training programme with a real project helps to close this gap.

This report is about my industrial training at AccioJob Pvt. Ltd. from 8 August to 18 March, done as part of my B.Tech CSE (Data Science) degree at SGT University. During the training I studied HTML, CSS, JavaScript, React, AI, AI agents, a little Node.js and the basics of MCP. As my training project, I built AuditPilot, an AI website audit agent.

1.2 Overview of MERN Stack Development

MERN is a popular JavaScript stack. The name comes from MongoDB, Express.js, React.js and Node.js. MongoDB is a document database. Express.js is a framework for building server routes and APIs. React.js is a library for building user interfaces from small reusable components. Node.js lets us run JavaScript on the server. Together they can be used to build a complete web application with a screen for the user, a server and a database.

Frontend development is the part that users see and click. HTML gives the structure, CSS controls the look, JavaScript adds behaviour and React organises the screen into components. Backend development handles requests, rules, login and the database. The two sides usually talk to each other through HTTP APIs and send data in JSON format.

Two newer ideas are also part of this report. An **AI agent** is a program in which a language model decides what to do next and can call tools to get facts or do work. **MCP** (Model Context Protocol) is an open standard that lets AI applications use tools in a common way. My project uses both ideas.

One point must be stated clearly. Express.js, MongoDB and JWT were not taught in my training. I learned them while building AuditPilot, with the help of an AI coding assistant. This is explained honestly in Chapter 3.

1.3 Need for Practical Training

Practical work builds habits that are hard to learn from theory alone. These include reading someone else's code, finding the cause of a bug, choosing clear names, keeping functions small and testing a change before calling the work done. It also teaches an improving mindset: build a small part, check the result, find a problem and fix it.

Modern web applications grow quickly. A small page can become a system with login, forms, tables, charts, background jobs and a database. A learner must understand not only each tool but also how the tools fit together. A project is the best way to learn this.

Training also prepares a student for teamwork. In a company, a task comes with requirements and limits. The developer asks questions, divides the task into steps, shares progress and writes down what is not finished. These habits are as important as technical knowledge.

1.4 Objectives of the Training

The main objective is to build a strong base in web development and then move towards full-stack development. The first goals are to build page structures with HTML, style responsive layouts with CSS, write interactive code with JavaScript and create reusable components with React.

A second objective is to understand AI and AI agents: how a language model answers, how it can use tools, and how to build an application around it in a safe way. A third objective is to complete one real project from idea to working software, including security, testing and documentation.

The training also aims to build professional habits such as planning work, reading documentation, debugging step by step and keeping code readable.

1.5 Scope of the Report

This report covers the training context, the topics I learned, the way I worked, the AuditPilot project (design, implementation and testing), the results I measured, the problems I faced and the limits of the work. It describes what is really completed. It does not claim that Express.js, MongoDB or JWT were taught in the classroom.

Company-specific details such as mentor names, attendance, marks and certificates are not invented in this report. Where an official detail is needed, it is left as a highlighted blank to be filled from the real training records.

1.6 Learning Outcomes Expected

By the end of the training I expected to understand the full development cycle: reading requirements, planning the design, writing the code, testing it, handling data and presenting the result. I also expected to be able to explain my choices in the project and to say clearly where the work has limits.

The technical path moves from frontend basics, through a small amount of Node.js, to a complete full-stack project that uses a database, login, an AI model and automated tests.

CHAPTER 2: SOFTWARE TRAINING WORK UNDERTAKEN

2.1 Training Context and Role

The training was held through AccioJob Pvt. Ltd. and ran from 8 August to 18 March. My role was that of a trainee. I attended the sessions, practised the topics, solved exercises and, at the same time, worked on my own project to apply what I learned.

The topics covered in the training were HTML, CSS, JavaScript, React, AI, AI agents, a little Node.js and the basics of MCP. Express.js, MongoDB and JWT were not part of the training, so I studied them separately for the project.

2.2 Learning Responsibilities

- Attend the sessions regularly and take notes.
- Practise every new topic with small programs before moving on.
- Read the official documentation when something is not clear.
- Break each task into small steps and finish them one by one.
- Use the new skills in a real project and keep the code in Git.
- Write down what is finished, what is not finished and what the limits are.

2.3 HTML Learning and Practice

HTML gives a web page its structure. I learned headings, paragraphs, links, images, lists, tables and forms. I also learned semantic tags such as main, nav and section, which tell the browser and screen readers what each part of the page means.

Accessibility was an important lesson. Images need alt text, form fields need labels, the page needs a language and headings must be in a sensible order. These are the same things that AuditPilot checks on other websites, so this topic is directly used in the project.

2.4 CSS Learning and Practice

CSS controls colour, spacing, fonts and layout. I practised the box model, Flexbox, Grid, media queries for different screen sizes and CSS variables for a consistent theme. I learned that a good layout is simple, has enough spacing and works on both a phone and a laptop.

In AuditPilot, CSS is used for the score gauge, the category bars, the cards, the severity badges (high, medium, low) and the responsive layout of the dashboard and report pages.

2.5 JavaScript Learning and Practice

JavaScript adds behaviour to a page. I learned variables, functions, arrays, objects, loops, array methods such as map and filter, modules, error handling, promises and async/await. I also learned how to use fetch to call an API and how JSON is used to send data.

JavaScript is used everywhere in AuditPilot: in the React screens, in the Express server and in the audit checks that read and test page HTML. Writing the same language on both sides made the project easier to manage.

2.6 React.js Learning and Practice

React builds the screen from components. I learned JSX, props, state, the useState and useEffect hooks, handling events, controlled form inputs, rendering lists with stable keys and page navigation with React Router. The main idea is that the screen is a function of the data: when the state changes, React updates the screen.

AuditPilot has pages for login, dashboard, report, history, compare and site audit. They use components such as the score gauge, finding cards and the chat panel. Recharts is used for the score trend chart.

2.7 AI and AI Agents

A large language model (LLM) writes text by predicting the next words. It is good at explaining, but it can make mistakes, so it should not be trusted for facts. I learned how prompts work, why the temperature setting matters and how to ask a model for short, structured answers.

An agent goes one step further. The model can ask to call a tool, such as “audit this URL”, receive the result and then write its answer. The program must check every tool request, limit the number of steps and handle errors. This lesson shaped the main rule of my project: tools compute the facts, the LLM only explains them.

2.8 Node.js and MCP Basics

Node.js runs JavaScript outside the browser. I learned how to run scripts, use npm packages and start a small server. I also learned the basics of MCP: a server lists its tools, and an AI application

such as Claude Code can call them. In AuditPilot, the same tool list is used by the chat agent and by an MCP server, so there is one source of truth.

2.9 Work Discipline and Problem Solving

Technical skill is only part of the job. I learned to read error messages fully, to reproduce a problem before changing code, to change one thing at a time and to write a test so that the same bug does not return. I also learned that being honest about limits, for example saying that a small model can be wrong, is better than hiding them.

CHAPTER 3: INDUSTRIAL TRAINING WORK UNDERTAKEN

3.1 Training Methodology

I followed a simple cycle: learn the idea, try it in a small example, use it in the project, test it and improve it. When something did not work, I looked at the official documentation first and then at examples.

Because Express.js, MongoDB and JWT were not covered in the training, I learned them while building the project. I used an AI coding assistant, Claude Code, for this. I want to be clear about how I used it. I wrote the requirements, took the design decisions (for example, “tools compute the facts, the LLM only explains”), reviewed the results, ran the tests and checked the real behaviour in the browser. The assistant wrote a large part of the code. I can explain the architecture, the flow and every feature of the project.

3.2 Daily Learning Workflow

My work on the project followed the same routine most days. I picked one small feature, wrote down what “done” would look like, built it, ran the automated tests, checked it in the browser and saved the work in Git with a clear commit message. If a test failed, I fixed the cause before moving on.

3.3 Task Planning and Execution

I divided the project into stages so that something worked at every step.

- **Stage 1:** design and rules, including the scoring rules and the security rules.
- **Stage 2:** the audit engine with 29 checks and the scoring code.
- **Stage 3:** the Express API, the database models, sign-up and sign-in.
- **Stage 4:** the AI agent, the tool registry, the MCP server and the local model.
- **Stage 5:** the React user interface: dashboard, report, compare and chat.
- **Stage 6:** automated tests and fixing the problems they found.
- **Stage 7:** extra features: history, scheduled audits, JavaScript rendering and the whole-site crawl.
- **Stage 8:** documentation and a one-command setup script for a new computer.

3.4 Debugging and Testing Practices

Many real problems appeared and each one taught something. When the demo database reset after a server restart, an old login still looked valid and the chat showed “Audit not found”. I fixed this by keeping the demo database on disk, checking that the user still exists on every request and showing a clear “session expired” message. When the small 3B model copied an audit id wrongly, I added short 8-character references and a way to find a finding by its title.

For testing I used Vitest and Supertest. The tests cover the checks, the scoring, the security guard, the login rules, the agent, the history, the crawler and the MCP server. The real screenshots in this report were taken from the running application with Playwright, not drawn by hand.

3.5 Communication, Feedback and Documentation

Good documentation makes a project usable by other people. I wrote a README with setup steps, settings, the API, the scoring rules and the known limits. I also wrote a demo script and a question-and-answer file to prepare for the presentation. The code is stored on GitHub, so it can be set up on another computer by running a single setup command.

3.6 Tools and Development Environment

Table 3.1 *— Tools used in the project*

Tool	Purpose
Node.js 22 and npm	Runs the server and installs packages
Express 5	Builds the REST API and serves the application
MongoDB and Mongoose	Stores users, audits, schedules and site audits
React 19, Vite and React Router	Builds the user interface and the pages
Recharts	Draws the score trend chart
Cheerio	Reads the HTML of the audited page for the checks
Ollama with qwen2.5:3b	Runs the open-source language model on my own laptop
Playwright and Vitest	Real screenshots, JavaScript rendering and automated tests
Git and GitHub	Version control and online backup of the code
Claude Code	AI coding assistant used during development

3.7 Professional Skills Developed

The project improved my planning, debugging and documentation skills. It also taught me security thinking: any program that fetches a user-given URL must be protected. I learned to design for failure, so that the app still works when the AI model is slow or offline. Most of all, I learned to measure results honestly and to report only numbers that I really measured.

CHAPTER 4: PROJECT WORK – AUDITPILOT: AI WEBSITE AUDIT AGENT

4.1 Project Introduction

AuditPilot is a web application that audits a website and explains the result. A signed-in user enters a URL. The application checks the page for SEO, accessibility, security and performance problems, gives a score out of 100 and shows the evidence for every finding. A local AI model then writes a short summary and a fix plan, and the user can ask follow-up questions in a chat.

The project follows one main rule: **tools compute the facts, the LLM only explains them.** The audit and the score are made by normal code and never wait for the AI. Because of this, the results appear in a fraction of a second, even if the AI model is slow or switched off.

The AI model runs on my own laptop through Ollama, so no cloud AI service, API key or payment is needed, and the audited data never leaves the machine.

4.2 Problem Statement

Website owners often do not know why their site ranks badly, is hard to use for some people or is open to attacks. Existing audit tools can be complex, can need paid accounts and usually show a long list of technical items without explaining what to fix first. Plain AI chatbots can explain things, but they can also invent facts about a site that they have not actually checked.

The problem is to build a tool that gives accurate, repeatable audit results and then uses AI only to explain those results in simple words. It must also be safe, because the server will fetch addresses given by users.

4.3 Objectives and Requirements

Functional requirements:

- Sign up and sign in with email and password.
- Audit any public web page with 29 checks in four categories and show evidence for each finding.
- Show a score for each category and an overall score.
- Write an AI summary and fix plan in the background, and answer questions in a chat that can call tools.

- Show history, compare two audits, export a report and run scheduled audits.
- Audit a whole site by crawling its pages in a polite way.

Non-functional requirements:

- The audit result must never depend on or wait for the AI model.
- Every fetch of a user-given URL must pass a security guard (SSRF protection).
- One user must never see another user's audits.
- No cloud AI service and no paid API; everything runs locally.
- The whole project must be easy to set up on a new computer.

4.4 System Design and Workflow

The browser talks to an Express API. The API handles login, audits, chat and the database. For a normal audit, the API calls the audit engine, which fetches the page through the safe fetcher, runs the checks and calculates the scores. This path has no AI in it. For the AI part, an agent loop sends the question to the local model through a queue, lets it call tools and streams the answer back to the browser.

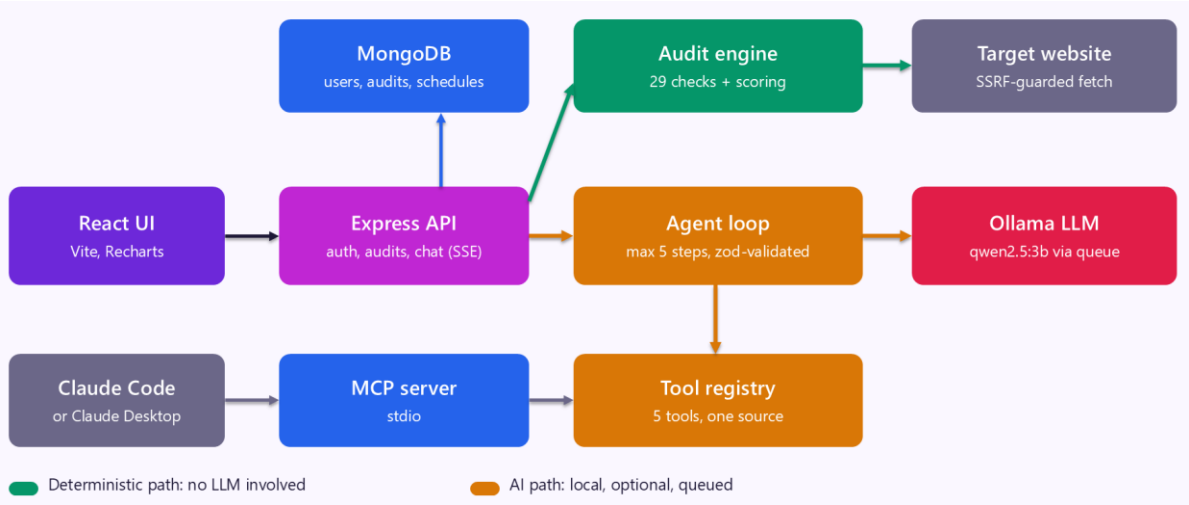


Figure 4.1 *— System Architecture of AuditPilot*

The same five tools are shared by the chat agent and by an MCP server. So Claude Code or Claude Desktop can also audit a site by using the project's tools. A scheduler runs timed audits, and an optional headless browser can render JavaScript pages, both behind the same security guard.

4.5 Audit Engine and Scoring

The audit engine uses Cheerio to read the HTML and checks the response headers and a few extra files such as robots.txt and sitemap.xml. It has 29 checks. Each check returns a result with a severity (high, medium, low or pass), the evidence and a fix recommendation. A static table of fix snippets gives ready-to-copy code, for example the CSP header for Express and for Nginx.

Table 4.1 *— The 29 audit checks*

Category	Checks	Examples
SEO	9	Title, meta description, H1, canonical link, Open Graph, viewport, robots.txt, sitemap.xml, broken links
Accessibility	5	Page language, image alt text, form labels, heading order, link and button names
Security	9	HTTPS, HSTS, CSP, X-Content-Type-Options, clickjacking protection, Referrer-Policy, Permissions-Policy, cookie flags, technology disclosure
Performance	6	Response time, HTML size, compression, Cache-Control, resource count, render-blocking scripts

The scoring is fully rule-based, so the same page always gets the same score. Each category starts at 100 and loses points for every problem. The overall score is a weighted average that gives the most weight to security.

Table 4.2 *— Scoring rules*

Item	Rule
Starting score of each category	100
High, medium and low finding	Minus 15, minus 7 and minus 3 points
Lowest possible category score	0
Overall score weights	Security 30%, SEO 25%, Accessibility 25%, Performance 20%

4.6 Security: SSRF Guard and Access Control

A tool that fetches any URL typed by a user can be misused to reach private parts of a network, such as a local database or a cloud metadata address. This attack is called SSRF (Server-Side

Request Forgery). Because of this risk, every fetch in the project goes through one guard. It does the following:

- Allows only http and https and no credentials inside the URL.
- Resolves the domain name and blocks loopback, private, link-local, multicast and cloud-metadata addresses, including IPv6.
- Checks the address again at connection time, so a DNS trick cannot swap the address after the first check.
- Allows at most 3 redirects and checks every redirect again.
- Stops a request after 10 seconds or 2 MB of data.

For access control, passwords are stored as bcrypt hashes, and the API uses JWT tokens. On every request the server also checks that the user still exists. All audits belong to one user, and another user's audit returns "not found", also for the AI tools. Other protections are helmet security headers, CORS, per-user rate limits and zod validation of every input and every tool call.

4.7 AI Agent, Tools and MCP

The AI model used is qwen2.5:3b, an open-source model that runs through Ollama on the CPU of my laptop. The agent loop works like this: the user's question and the data of the current audit are sent to the model. If the model asks for a tool, the program checks the arguments with zod, runs the tool and gives the result back to the model. This repeats up to 5 steps, and then the final answer is streamed to the browser word by word.

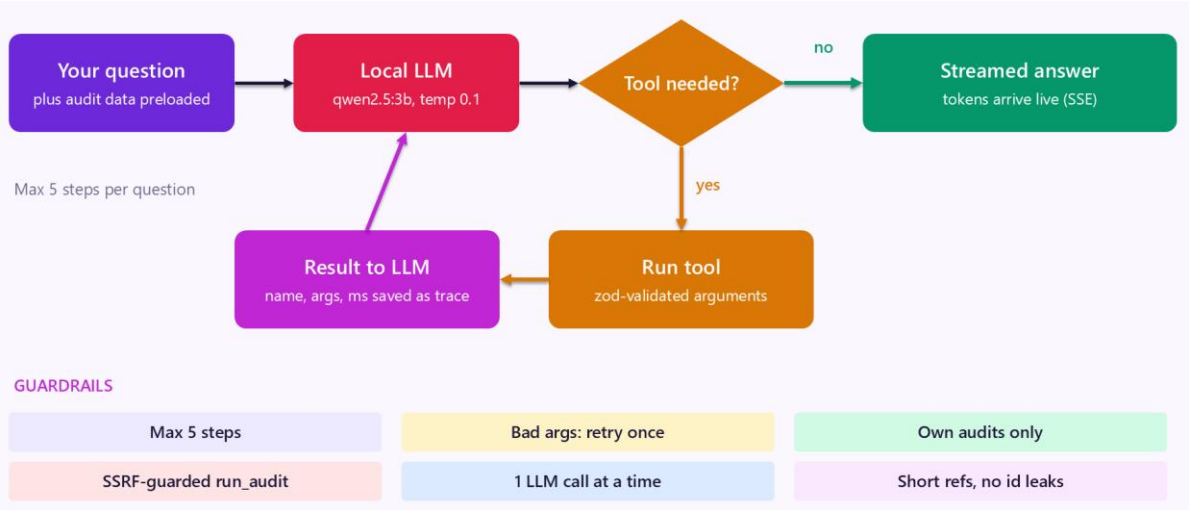


Figure 4.2 *— How the AI Agent Works*

Small models can make mistakes with tool arguments. To handle this, the program retries once with a helpful error message, uses short 8-character audit references instead of long ids, never lets the model choose a user id, and runs only one model call at a time through a queue, with priority for chat.

Table 4.3 *— Tools shared by the chat agent and the MCP server*

Tool in chat	Name in MCP	What it does
run_audit	audit_url	Audits a URL and saves the result
get_audit	get_audit	Returns the scores and findings of one saved audit
list_my_audits	list_audits	Lists the recent audits of the user
compare_audits	compare_audits	Compares two audits and lists fixed and new issues
get_remediation	get_remediation	Gives fix snippets for a finding

If the model is not available, the app shows “Local LLM offline” and the audits still work. A built-in mock model, clearly labelled “MOCK LLM”, is used only for tests and as an emergency fallback.

4.8 Database Design

MongoDB stores four collections. **User** keeps the email and the hashed password. **Audit** keeps one audit with its scores, its findings, the AI summary, the status of the AI job and the chat history. **Schedule** keeps the timed audits of a user (up to 5 per user). **SiteAudit** keeps a whole-site crawl with its pages, broken links, site-wide findings and scores. If no database address is given, the app uses a local file database so that it works without any setup.

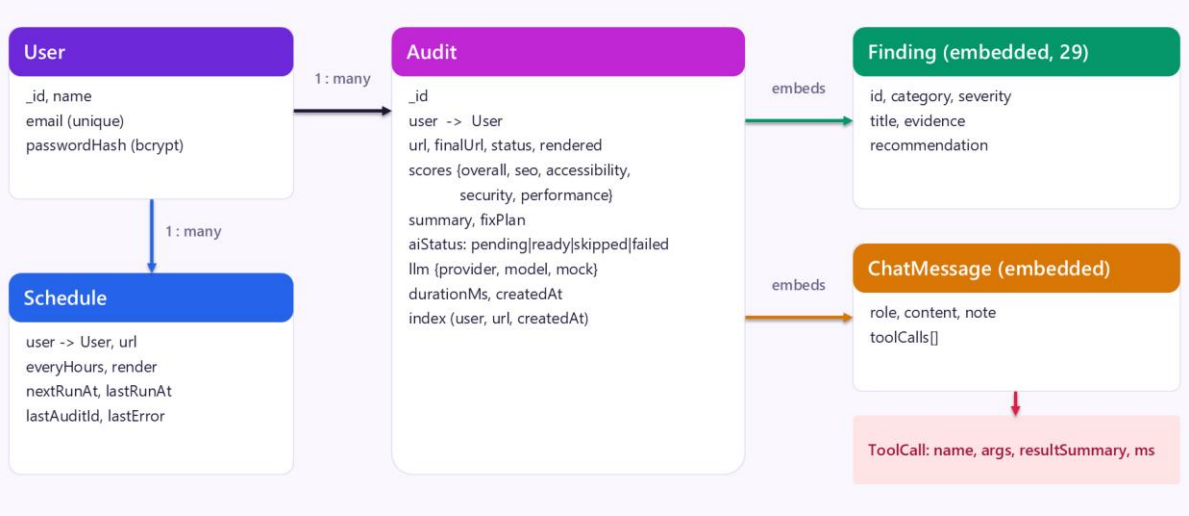


Figure 4.3 *— Data Model of AuditPilot (the SiteAudit collection for whole-site crawls is not drawn)*

4.9 Frontend Implementation

The user interface is built with React, Vite and React Router. The dashboard has the URL form, quick-audit buttons, a score trend chart, recent audits and the schedule panel. The report page shows an animated score gauge, four category scores, an AI summary and fix plan, every finding with evidence and ready-to-copy fix code, and the chat panel.

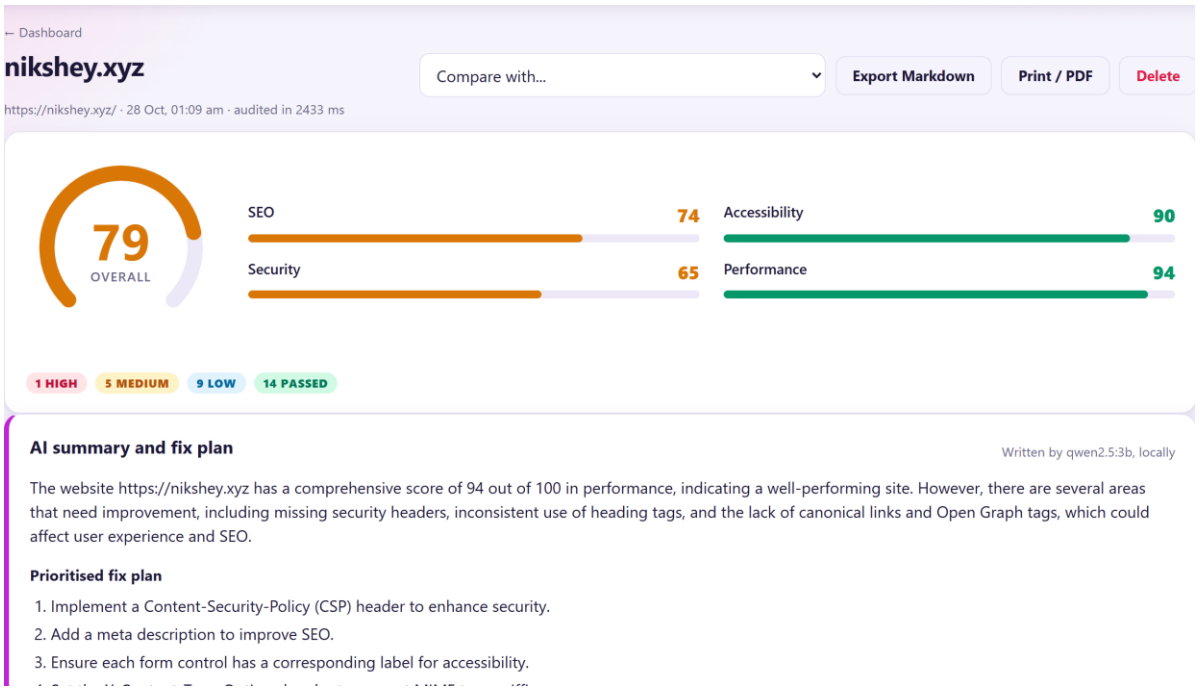


Figure 4.4 *— Audit Report of a Real Website with Scores and AI Summary*

Each finding can be opened to see the evidence and a fix snippet, for example the Express and Nginx code for a Content-Security-Policy header.



Figure 4.5 *— Evidence and Fix Snippet for a Finding*

The chat panel lets the user ask questions. When the agent uses a tool, a collapsible tool trace shows the tool name, its arguments, the result and the time taken.

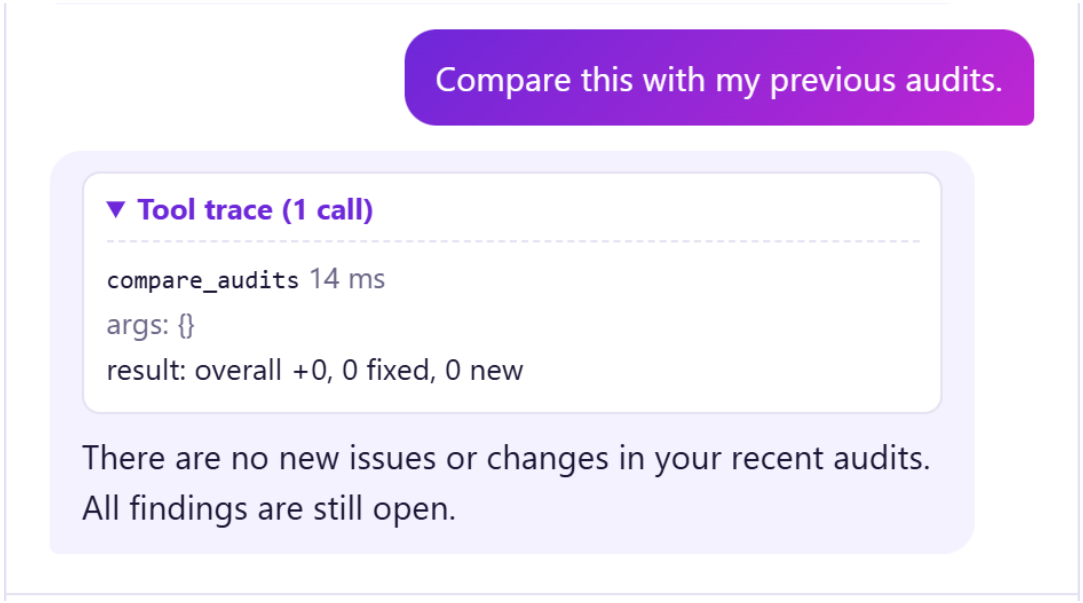


Figure 4.6 *— Agent Chat with a Tool Trace*

The compare page shows two audits side by side, with the score change in every category and the list of issues that were fixed or are new.

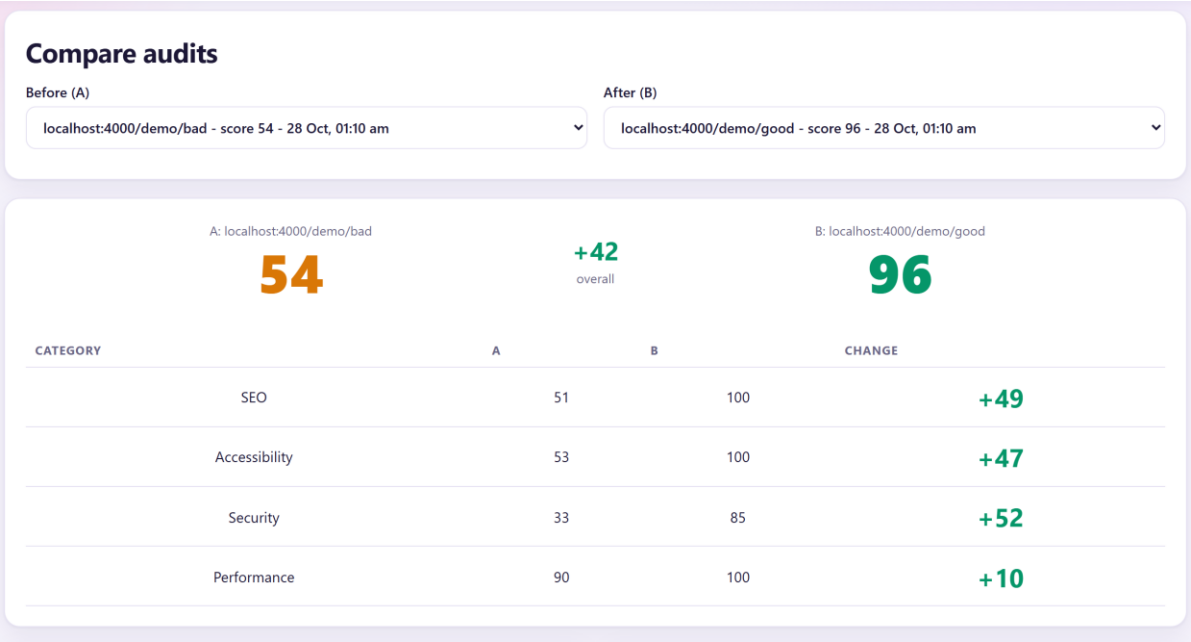


Figure 4.7 *— Comparing the Bad Demo Page with the Fixed Page*

The history page lists every audit with its score, number of issues, AI status and saved chat size. It supports search by URL, paging, delete and selecting two audits to compare.

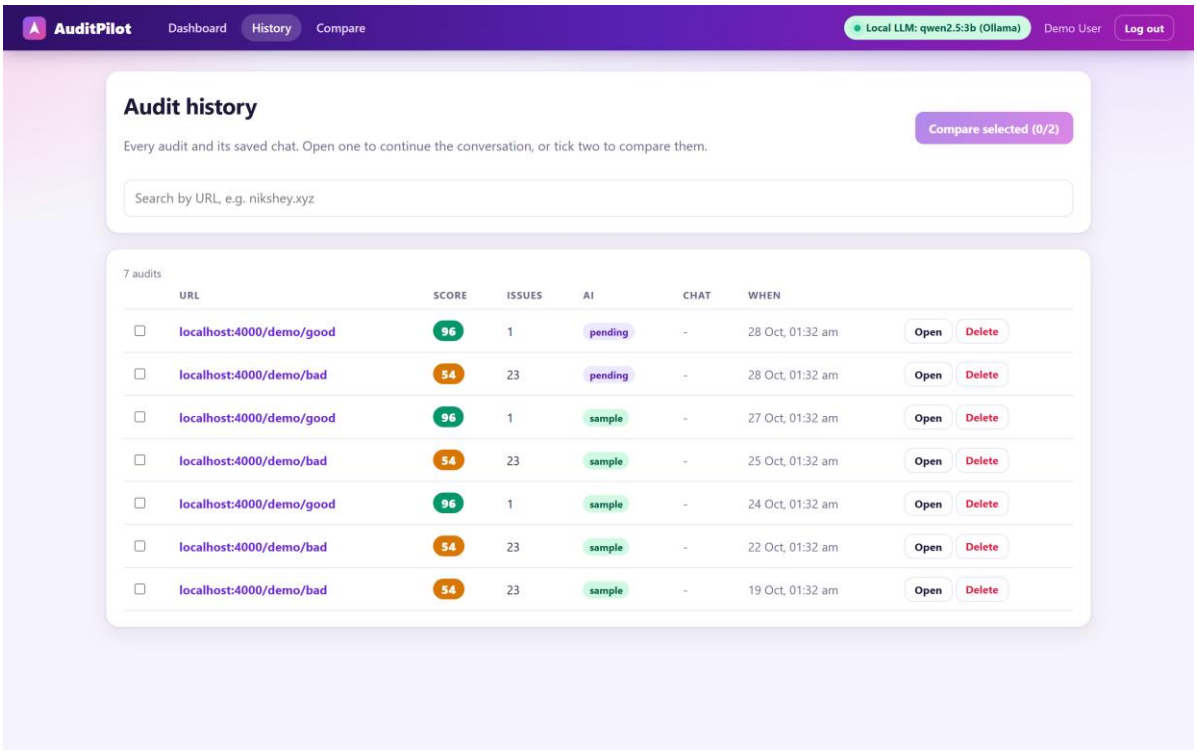


Figure 4.8 *— History Page*

4.10 Extra Features

- **Sign-up and sign-in:** accounts with email and password, a show/hide password option and a clear message when a session has expired.
- **Scheduled audits:** a URL can be re-audited every few hours by an in-process scheduler (up to 5 schedules per user).
- **JavaScript rendering:** an optional headless browser (the installed Chrome or Edge, through Playwright) renders pages that are built by JavaScript. Every request of this browser also passes the SSRF guard.
- **Export:** a report can be exported as Markdown or printed.
- **One-command setup:** setup.bat (Windows) or setup.sh (macOS and Linux) installs the tools, downloads the model and creates the settings file, also on a computer with an NVIDIA GPU.

4.11 Whole-Site Audit (Crawl)

A single page does not show the health of a whole site, so I added a site audit. The user ticks “Audit the whole site” and chooses up to 5, 10 or 25 pages. A background job crawls the pages of the same website, starting from the given page, and every page becomes a normal audit with its own report and chat.

The crawler is written to be polite and safe:

- It obeys robots.txt and identifies itself as AuditPilotBot.
- It also uses the pages listed in sitemap.xml, but only from the same website.
- It waits between requests and stops at the page limit.
- It stays on the same website: a page that redirects to another site is never audited.
- Every request goes through the SSRF guard, and a user can run only one crawl at a time.

After the crawl, extra site-wide checks look for problems that a single page cannot show: duplicate page titles, duplicate meta descriptions and broken internal pages. The site score is the average of the page scores in each category, minus points for the site-wide findings. One AI summary is written for the whole site.

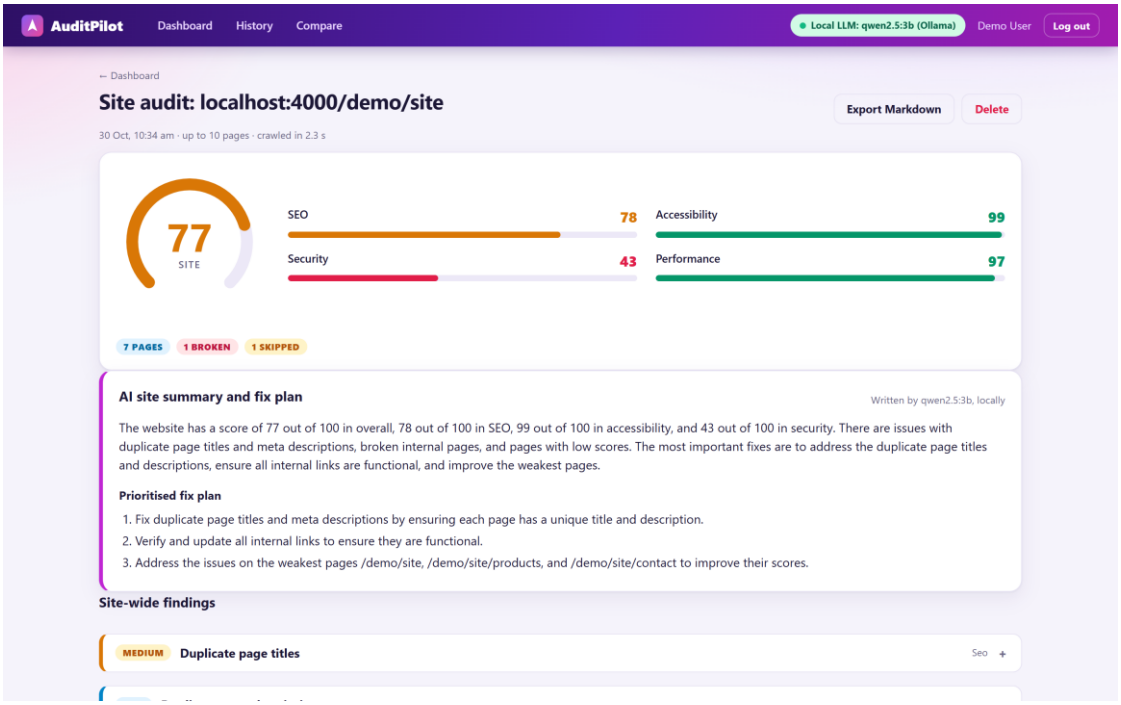


Figure 4.9 *— Whole-Site Audit with Site Score and AI Summary*

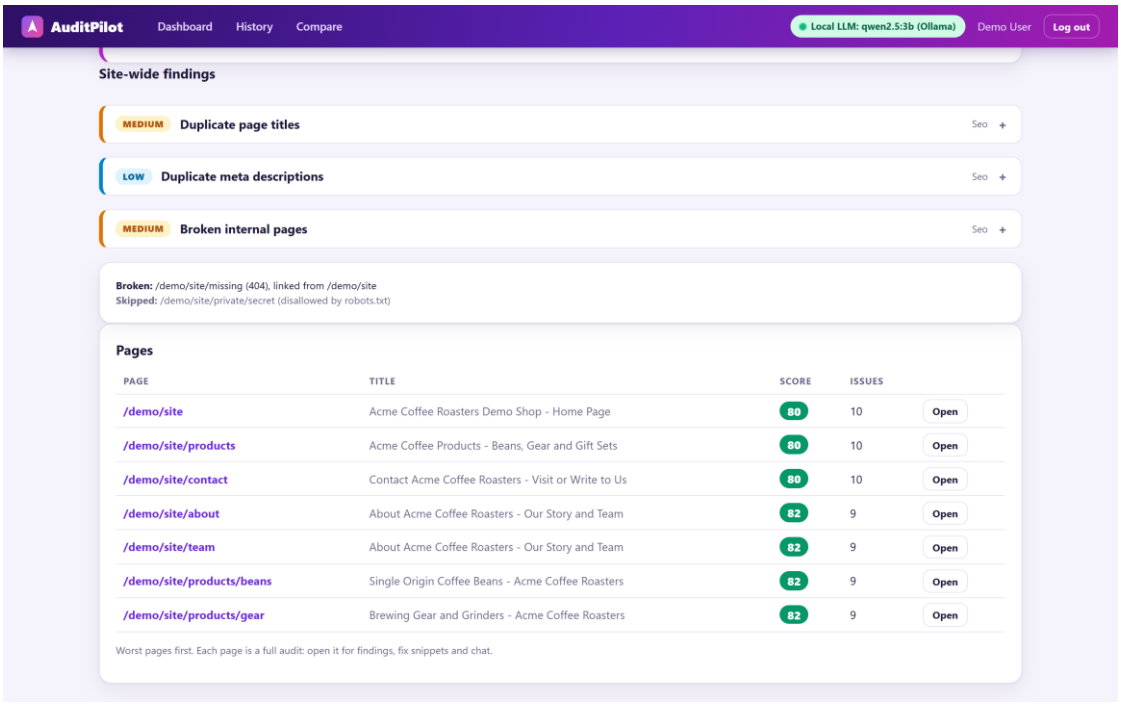


Figure 4.10 *— Site-Wide Findings and the Page List*

4.12 Testing and Validation

The project has 104 automated tests, and all of them pass. They run with one command, npm test. The tests use a mock model, so they are fast and do not depend on the real AI model. A separate opt-in set of 4 tests runs against the real local model.

***Table 4.4** — Automated tests by area*

Area	Tests
Checks, scoring and compare	19
SSRF guard	21
Login, API and ownership	10
Agent, summary and chat	18
Rendering, schedules, short references and links	8
History, sessions, sign-up and sign-in	7
Site crawl, robots.txt, sitemap and site-wide checks	18
MCP client test	3
Total	**104**

4.13 Limitations and Project Learning

- A 3B model on a CPU takes about 15 to 40 seconds for an answer and can be imprecise. This is why scores never depend on it.
- JavaScript rendering needs Chrome or Edge to be installed and is slower.
- The crawl finds pages from links and /sitemap.xml only, stops at 25 pages, and a crawl stopped by a server restart is marked as failed.
- The local demo pages use HTTP, so even the fixed page loses points on the HTTPS check.
- Scheduled audits run only while the server is running.

The main learning from the project is that a reliable design separates facts from explanation. Normal code is used where the answer must be exact, and the AI is used only where explanation is needed.

CHAPTER 5: RESULTS AND DISCUSSIONS

5.1 Training Outcomes

The training gave me a practical base in frontend development and an introduction to AI agents. HTML, CSS, JavaScript and React were used directly to build the user interface. The lessons on AI agents and MCP became the core of the project. The small amount of Node.js I learned was enough to start the server work, which I then extended by self-study.

Another outcome is a better way of debugging: reproduce the problem, check the inputs, read the error fully, change one thing and write a test. This habit saved a lot of time in the project.

5.2 Project Results

All numbers below were measured from the running application or from the automated tests. They were not estimated. The first table shows the scores of the two demo pages that are built into the project. The “bad” page has many deliberate problems and the “good” page is the same page after the fixes.

Table 5.1 *— Scores measured by AuditPilot*

Page audited	Overall	SEO	Accessibility	Security	Performance
Demo bad page (before fixes)	54	51	53	33	90
Demo good page (after fixes)	96	100	100	85	100
Real website (nikshey.xyz)	79	74	90	65	94
Whole-site audit of the demo site (7 pages)	77	78	99	43	97

Fixing the demo page raised the overall score from 54 to 96, a gain of 42 points. The comparison shows 22 fixed issues, no new issue and one issue still open, which is the HTTPS check, because the local demo pages run on plain HTTP. The whole-site audit of the demo site crawled 7 pages, found 1 broken link, skipped 1 page blocked by robots.txt, and reported duplicate titles and duplicate descriptions.

***Table 5.2** — Speed measured on my laptop (CPU only)*

Action	Time
Audit of a local page (no AI)	Tens of milliseconds
AI summary and fix plan	About 20 to 35 seconds
Chat answer	About 15 to 40 seconds
Crawl of the 7 demo pages	About 2 seconds
Whole-site audit including the AI summary	About 1 minute

5.3 Technical Observations

Separating the audit from the AI was the most useful design choice. Scores are exact and repeat the same way every time, and the app stays useful when the model is slow or off. The AI summary is created in the background, so the page opens at once and the summary appears when it is ready.

Streaming the answer word by word makes a slow model feel much faster. Checking every tool argument with zod and limiting the agent to 5 steps stopped the small model from causing errors or endless loops. Using one tool list for both the chat agent and the MCP server avoided duplicate code.

For the crawler, reusing the same safe fetcher meant that the SSRF protection did not need to be written again. Sharing one cached probe between all pages of a crawl avoided repeating the same requests for robots.txt and similar files.

5.4 Challenges and Solutions

Table 5.3 *— Main challenges and how they were solved*

Challenge	Solution
The small 3B model made mistakes in tool arguments, such as a wrong audit id	Short 8-character references, optional arguments, helpful error messages, one retry and lookup of a finding by its title
The local model is slow on a CPU	Audit and score never wait for it; the summary is generated in the background through a queue; the model is warmed up when the server starts
A reset demo database left an old login and showed “Audit not found”	A demo database stored on disk, a check that the user exists on every request and a clear “session expired” message
Pages built by JavaScript look empty to a normal fetch	An optional headless browser, with every request checked by the SSRF guard
Crawling a whole site safely	robots.txt rules, same-origin only, off-site redirects refused, delay between pages, a page limit and one crawl per user
Setting up the project on a new computer	A one-command setup script that installs the tools, downloads the model and creates the settings file

5.5 Skills Gained and Self-Assessment

I gained skills in planning a project in stages, writing and testing a REST API, building a React interface, designing a safe fetcher, working with a local AI model and writing documentation. I also learned to explain a technical design in simple words.

I want to give a realistic self-assessment. I completed the frontend topics and the introduction to AI agents and MCP in the training. Express.js, MongoDB and JWT I learned through self-study with an AI assistant while building the project, so my experience in them is that of a first project, not of a long professional career. I understand the design of AuditPilot and can explain it, and I am still improving in depth.

5.6 Limitations and Improvement Opportunities

The limits are listed in Section 4.13. In addition, the project is a learning project and not a production service. It has no email alerts, no payment or team features and it has been tested

mostly on my own laptop and on the demo pages. The automated tests give good confidence, but real users on many different websites would find more cases.

The checks are a practical set of 29 rules, not a replacement for full tools such as Lighthouse or a complete security test. The AI summary can be wrong sometimes, which is why the evidence for every finding is always shown beside it.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The training at AccioJob Pvt. Ltd. gave me a base in HTML, CSS, JavaScript and React, and an introduction to AI, AI agents, Node.js and MCP. I used these lessons to build AuditPilot, a complete full-stack project with a React interface, an Express API, a MongoDB database, a safe audit engine and a local AI agent.

The project follows one clear idea: tools compute the facts and the language model only explains them. Because of this, the audit is fast, exact and repeatable, and the AI adds value without being able to invent the results. The project also includes real security work (an SSRF guard, login, access control), 104 passing automated tests, history, scheduled audits, JavaScript rendering and a polite whole-site crawler.

The report has described the work honestly, including what was not taught in the training, how an AI assistant was used and where the limits are. I consider the project a successful learning project and a good base for further work.

6.2 Future Scope

The first improvement is a better AI layer. A larger or fine-tuned model, or a GPU, would give faster and more precise answers. Retrieval from the WCAG and OWASP documents could help the model give more accurate advice.

Next, the app can be made more useful for teams: email alerts for scheduled audits, a CI/CD option that fails a build when the score is low and a deployed hosted version with a managed database.

The audit itself can grow with more checks and Lighthouse-style performance metrics. The crawler can be extended with more sitemap locations and a higher page limit. Finally, more automated tests and testing on many real websites will make the project stronger.

APPENDIX A: PROJECT FEATURE CHECKLIST

This appendix lists the features described in Chapter 4. It is supporting material and does not add any new chapter.

- Sign up and sign in with email and password; sessions are checked on every request.
- Audit a URL with 29 checks in SEO, accessibility, security and performance.
- Show evidence, severity and a ready-to-copy fix snippet for every finding.
- Calculate category scores and a weighted overall score with fixed rules.
- Write an AI summary and fix plan in the background; show its status.
- Chat with an agent that can call tools, and show a collapsible tool trace.
- Save the chat history for each audit and allow it to be cleared.
- Keep a history of all audits with search, paging and delete.
- Compare two audits and list fixed, new and unchanged issues.
- Show a score trend chart for each URL on the dashboard.
- Export a report as Markdown or print it.
- Schedule repeated audits of a URL (up to 5 per user).
- Render JavaScript pages with an optional headless browser.
- Block unsafe URLs with the SSRF guard, also for the headless browser and the crawler.
- Audit a whole site with a polite same-origin crawl (robots.txt, sitemap.xml, page limit).
- Find duplicate titles, duplicate descriptions and broken internal pages across a site.
- Show a site score and one AI summary for the whole site.
- Share the same five tools between the chat agent and an MCP server.
- Work without the AI model: audits and scores still appear instantly.
- Set up on a new computer with one command.

APPENDIX B: TEST CASES

These test cases were checked on the running application or are covered by the automated tests.

Audit the bad demo page

Action: Click Audit demo (bad).

Expected result: The report opens at once and shows an overall score of 54.

Audit the fixed demo page

Action: Click Audit demo (good).

Expected result: The report shows an overall score of 96.

Compare two audits

Action: Open the Compare page and select the bad and the good audit.

Expected result: The page shows +42 overall and 22 fixed issues.

Block a private address

Action: Try to audit http://169.254.169.254/ with the safety guard on.

Expected result: The request is refused by the SSRF guard.

Wrong password

Action: Sign in with a wrong password.

Expected result: An error message is shown and no access is given.

Ownership

Action: Open the audit of another user by its id.

Expected result: The server answers “not found”.

AI model off

Action: Stop Ollama and run an audit.

Expected result: The audit and the scores still appear; the header shows that the local LLM is offline.

Chat tool call

Action: Ask: Compare this with my previous audits.

Expected result: The tool trace shows the compare_audits tool with its time.

History search

Action: Type demo/good in the history search box.

Expected result: Only the matching audits are listed.

Whole-site audit

Action: Audit the demo site with the crawl option.

Expected result: 7 pages are audited, 1 broken link is found and the robots.txt-blocked page is skipped.

Sitemap pages

Action: Crawl a site whose sitemap lists pages that are not linked.

Expected result: The sitemap pages are audited, and sitemap pages that return 404 are reported as broken.

APPENDIX C: TECHNOLOGIES AND REFERENCES

The following official documents and websites were used for learning and for building the project.

- MDN Web Docs: HTML, CSS and JavaScript (developer.mozilla.org).
- React documentation (react.dev).
- Node.js documentation (nodejs.org).
- Express.js documentation (expressjs.com).
- MongoDB and Mongoose documentation (mongodb.com, mongoosejs.com).
- Ollama documentation and model library (ollama.com).
- Model Context Protocol documentation (modelcontextprotocol.io).
- OWASP: Server-Side Request Forgery Prevention Cheat Sheet (owasp.org).
- W3C Web Content Accessibility Guidelines, WCAG (w3.org/WAI).
- Playwright and Vitest documentation (playwright.dev, vitest.dev).

Project source code: https://github.com/NiksheyYadav/AuditPilot (the README explains the setup, the settings, the API and the scoring rules).